Frontend · concept board

HTML

Structure, semantics, forms, accessibility and SEO basics

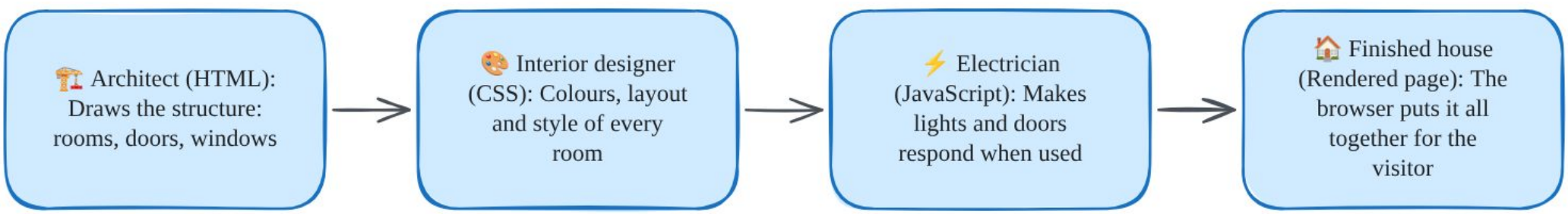
1. What is HTML?
2. Semantic HTML
3. Block vs inline, and common elements
4. Forms and validation
5. Accessibility (a11y)
6. The <head>, SEO and loading scripts
7. Browser storage

Draw on any slide, then Present (arrow keys) or export the board as a PDF.

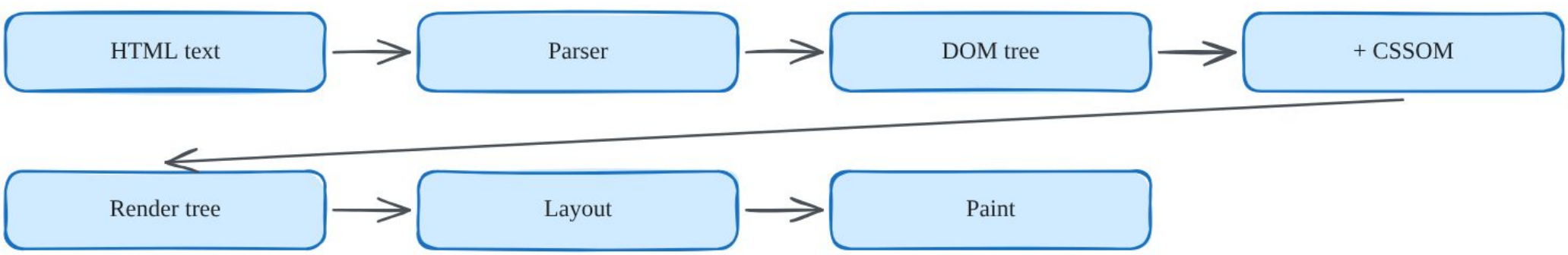
1. What is HTML?

Definition

HTML (HyperText Markup Language) describes the structure and meaning of a web page using elements (tags).
The browser parses it into the DOM tree, CSS styles that tree and JavaScript makes it interactive.



Think of an HTML page like building a house



How a page becomes pixels (the critical rendering path)

Minimal valid page

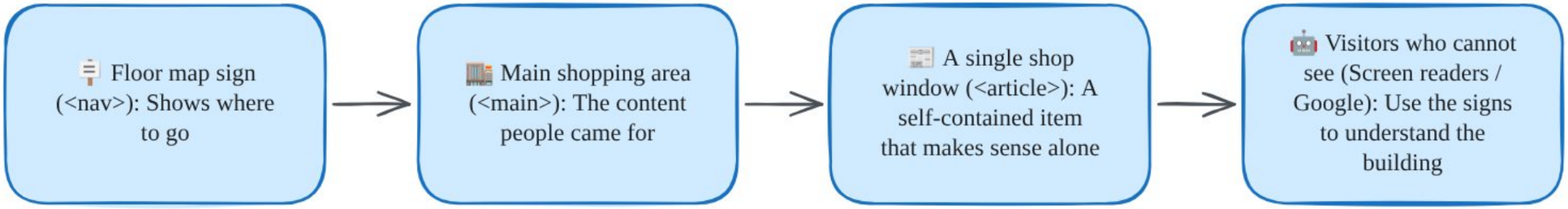
```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>My page</title>
  </head>
  <body>
    <h1>Hello, world</h1>
    <p>A paragraph with a <a href="https://developer.mozilla.org">link</a>.</p>
  </body>
</html>
```

- <!DOCTYPE html> switches the browser to standards mode (no quirks).
- lang helps screen readers and translation.
- The viewport meta tag makes the page responsive on phones.

2. Semantic HTML

Definition

Semantic elements describe what content is, not how it looks. They improve accessibility, SEO and readability.



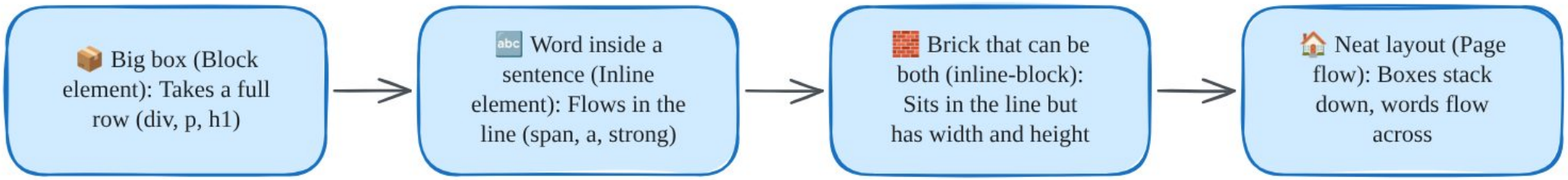
Think of semantic HTML like signs in a shopping mall

Element	Use for
<header> / <footer>	Intro / closing content of a page or section
<nav>	Main navigation links
<main>	The unique main content (one per page)
<article>	Self-contained content: a post, a product card
<section>	A themed group of content with a heading
<aside>	Side content: related links, ads
<figure> + <figcaption>	Image/diagram with a caption
<button>	Actions (not a <div onclick>)

Common mistake

"Div soup": is not focusable and is not announced as a button. Use <button>; use <a> only for navigation to a URL.

3. Block vs inline, and common elements

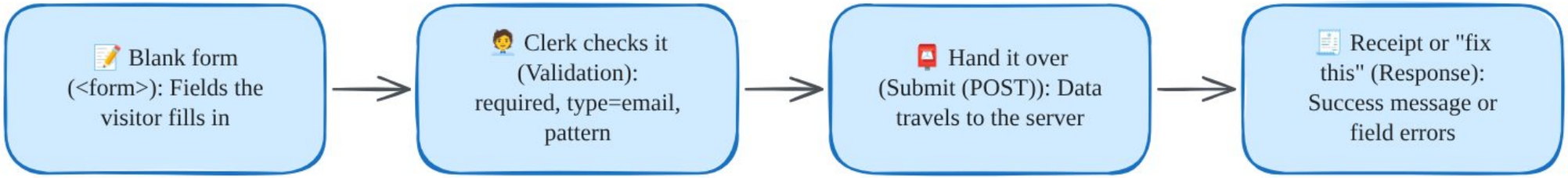


Think of block vs inline like moving boxes and words

Block	Inline
Starts on a new line, takes full width	Flows inside text, only as wide as content
div, p, h1–h6, ul, section, form	span, a, strong, em, img, code, label

- = important (bold), = emphasis (italic) — semantic; <b>/<i> are only visual.
- <img src alt> — alt is required for accessibility; use alt="" for decorative images.
- Lists: <ul> unordered, <ol> ordered, <li> items, <dl> description list.
- Tables for tabular data only: <table> <thead> <tbody> <tr> <th scope="col"> <td>.

4. Forms and validation



Think of a web form like a paper application at a bank

Accessible form with built-in validation

```
<form id="signup" novalidate>
  <label for="email">Email</label>
  <input id="email" name="email" type="email" required autocomplete="email" />

  <label for="age">Age</label>
  <input id="age" name="age" type="number" min="18" max="99" />

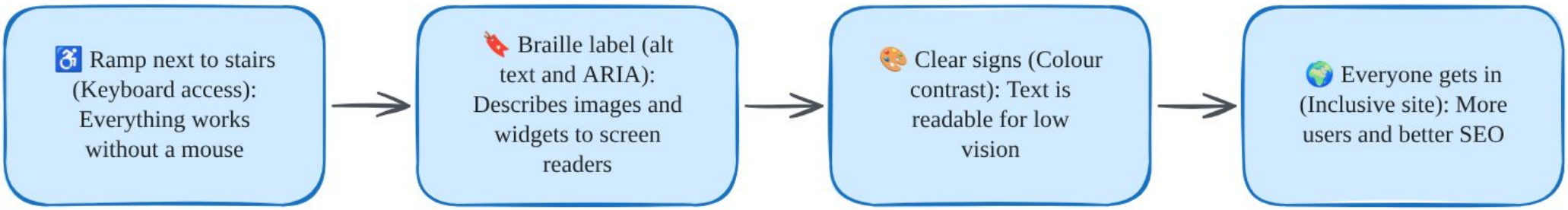
  <label><input type="checkbox" name="terms" required /> I accept the terms</label>
  <button type="submit">Sign up</button>
</form>
<p id="msg"></p>
<script>
  document.getElementById('signup').addEventListener('submit', (e) => {
    e.preventDefault()
    const form = e.target
// … 5 more lines in the notes
```

- Every input needs a <label> (linked with for/id or by wrapping).
- Input types (email, number, date, tel, url) give free validation and the right mobile keyboard.
- Attributes: required, min/max, minlength, pattern, autocomplete.
- GET puts data in the URL (search, filters); POST sends it in the body (create, login).

Common mistake

Client-side validation is for UX only — always validate again on the server.

5. Accessibility (a11y)

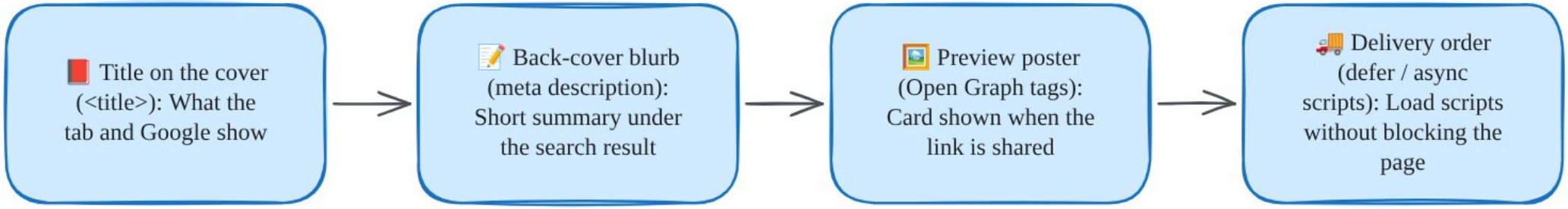


Think of accessibility like a building with ramps and braille

- Use semantic elements first; ARIA only fills gaps ("no ARIA is better than bad ARIA").
- One <h1>, then headings in order (don't skip levels for styling).
- Everything clickable must work with the keyboard (Tab, Enter, Space) and show a visible focus ring.
- Text contrast at least 4.5:1; never use colour alone to carry meaning.
- aria-label names an icon-only button; aria-live announces dynamic updates.

> **Tip**
>
> Quick test: unplug the mouse and use the page with Tab only, then run Lighthouse / axe.

6. The <head>, SEO and loading scripts



Think of the <head> like a book cover and back page

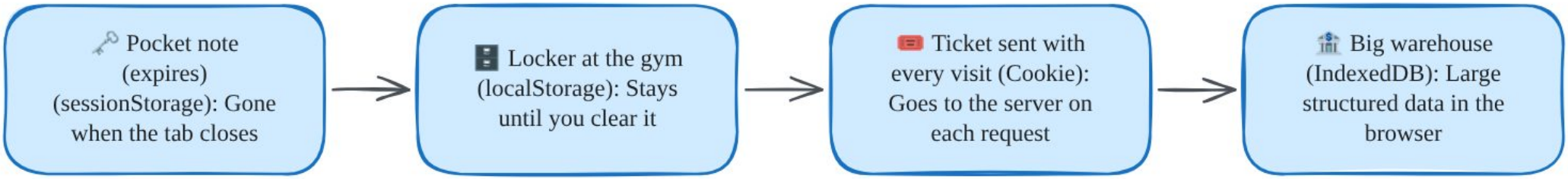
- <title> and <meta name="description"> show in search results.
- Open Graph tags (og:title, og:image) control link previews on social media.
- <link rel="canonical"> avoids duplicate-content issues.
- Search engines understand semantic HTML and real links (<a href>).

Script tag	Downloads	Runs	Order kept?
<script>	blocks parsing	immediately	yes
<script defer>	in parallel	after HTML is parsed	yes
<script async>	in parallel	as soon as downloaded	no
<script type="module">	in parallel	deferred by default	yes

Tip

Use defer for app scripts, async for independent ones (analytics). Add loading="lazy" on below-the-fold images.

7. Browser storage



Think of browser storage like places to keep your things

	Capacity	Lifetime	Sent to server?
Cookie	~4 KB	Until expiry	Yes, on every request
localStorage	~5–10 MB	Until cleared	No
sessionStorage	~5 MB	Until the tab closes	No
IndexedDB	Large (quota)	Until cleared	No

Common mistake

Don't keep auth tokens in localStorage — any XSS can read them. Prefer httpOnly, Secure, SameSite cookies.

Revise

HTML = structure + meaning. Semantic tags, labelled forms, keyboard access, defer scripts, and the right storage for each job.

Rapid-fire Q&A (1/2)

What does <!DOCTYPE html> do?

Tells the browser to render in standards mode instead of quirks mode.

Semantic vs non-semantic elements?

Semantic elements (header, nav, main, article, button) describe meaning; div and span carry none. Semantics help accessibility, SEO and maintenance.

Block vs inline elements?

Block elements start on a new line and fill the width (div, p, h1). Inline elements flow in text and only take their content width (span, a, strong).

async vs defer on script tags?

Both download in parallel. defer runs after parsing, in order. async runs as soon as it downloads, in any order.

Why is the alt attribute important?

Screen readers read it, it shows if the image fails, and search engines use it. Decorative images get alt="".

localStorage vs sessionStorage vs cookies?

localStorage persists until cleared; sessionStorage lasts for the tab; cookies are small and are sent to the server on every request.

Rapid-fire Q&A (2/2)

How do you make a form accessible?

A label for every input, correct input types, clear error messages linked to fields, keyboard operable, and visible focus.

What is the DOM?

The Document Object Model: the browser's tree of objects built from HTML that JavaScript can read and change.

When should you use ARIA?

Only when native HTML can't express the role or state. Native elements first; ARIA fills gaps.

What is the critical rendering path?

HTML → DOM, CSS → CSSOM, combined into the render tree, then layout and paint. Blocking CSS/JS delays the first paint.